# SOFTWARE ENGINEERING AND DESIGN PRINCIPLES

## ( CS311L )

Name: Anupriya Saxena RollNo: 202401100200070
Branch Sec: CSE 5A Lib ID: 2428cse616

# AI CHATBOT FOR COLLEGE HELPDESK

## Software Metrics & Complexity Analysis

### 1. Aim

To analyze the software metrics and complexity of the AI Chatbot for College Helpdesk by identifying suitable size, complexity, quality, and maintainability metrics. The aim is to understand the effort required to develop and maintain the system and to evaluate its major workflows using software engineering measurement techniques.

### 2. Introduction

Software metrics are quantitative measures used to evaluate the size, complexity, quality, performance, and maintainability of a software system. They help developers understand the characteristics of a system and identify areas that may require improvement.

The AI Chatbot for College Helpdesk is designed to provide information related to admissions, examinations, fee payments, timetable, hostel, library, placements, and other college services. It supports general queries without login and requires authentication before displaying personalized academic or fee information. Unresolved queries can be transferred to a human helpdesk operator through a support ticket.

Software metrics can help analyze this system by measuring the complexity of its query-handling workflow, the size of its functional requirements, and the quality of its responses and services.

### 3. Objectives

The main objectives of this assignment are:

- To understand the importance of software metrics in software development.
- To identify suitable metrics for the AI Chatbot for College Helpdesk.
- To analyze the complexity of the query-handling process.
- To calculate the cyclomatic complexity of a representative workflow.
- To understand Function Point Analysis for estimating software size.
- To identify quality metrics such as reliability, maintainability, performance, and usability.
- To interpret software measurements and identify possible areas for improvement.

### 4. Software Metrics for the AI Chatbot

The following metrics are relevant to the AI Chatbot for College Helpdesk.

| Metric | Description | Application in the project |
|---|---|---|
| Lines of Code (LOC) | Measures the number of lines of source code. | Estimates implementation size once the source code is available. |
| Cyclomatic | Measures the number of linearly | Evaluates the complexity of query handling and |

| | | |
|---|---|---|
| Complexity | independent paths through a program. | authentication logic. |
| Function Points (FP) | Measures software size based on user-visible functionality. | Estimates the functional size of the chatbot. |
| Response Time | Measures the time taken to generate a response. | Evaluates how quickly the chatbot answers user queries. |
| Reliability | Measures the ability of the system to operate without failure. | Evaluates availability of chatbot services and integrations. |
| Maintainability | Measures how easily software can be modified or corrected. | Evaluates the modularity of the chatbot components. |
| Defect Density | Measures defects relative to software size. | Helps evaluate implementation quality after testing. |
| Test Coverage | Measures the proportion of code or functionality covered by tests. | Checks whether query handling, authentication, and ticket workflows are adequately tested. |

## 5. Size and Complexity Analysis

The chatbot consists of multiple functional components. Each component performs a specific responsibility and contributes to the overall complexity of the system.

| Component | Main functionality | Relative complexity |
|---|---|---|
| Chat Interface | Accepts queries and displays responses. | Low |
| NLP Engine | Identifies user intent and extracts entities. | High |
| Authentication Service | Verifies users through login or OTP. | Medium |
| Knowledge Base Service | Searches FAQs and college information. | Medium |
| ERP/SIS Integration | Retrieves academic and fee information. | High |
| Ticket Service | Creates, assigns, and updates support tickets. | Medium |
| Admin Dashboard | Manages conversations, tickets, and knowledge-base entries. | Medium |
| Notification Service | Sends ticket confirmations and status updates. | Low to Medium |

The NLP engine and ERP/SIS integration may be relatively complex because they involve query interpretation, external system communication, data validation, and error handling. The chatbot's overall complexity can be reduced by dividing these responsibilities into separate modules.

### 5.1 Lines of Code (LOC)

Lines of Code is a software size metric that counts the lines of source code used to implement a program.

\[ LOC = \text{Total number of source code lines} \]

LOC can help estimate the size of the chatbot implementation. However, the exact LOC cannot be determined from the supplied SRS and diagrams because the complete source code is not available.

For illustration, suppose a future implementation contains 1,200 lines of source code. The estimated size would be:

\[ LOC = 1200 \]

This is only an example value, not a measured result from the actual project.

### 5.2 Advantages of LOC

- It is simple to understand and calculate.
- It can help estimate implementation size.
- It can be used to calculate defect density when defect and LOC data are available.

### 5.3 Limitations of LOC

- Different programming languages require different numbers of lines for the same functionality.
- It cannot directly measure the quality or usefulness of software.
- It is difficult to estimate accurately before implementation.

## 6. Cyclomatic Complexity Analysis

Cyclomatic Complexity is a software metric introduced by Thomas J. McCabe. It measures the number of linearly independent execution paths in a program's control-flow graph.

It helps identify the complexity of conditional logic and estimate the minimum number of independent paths that should be considered during testing.

The formula is:

\[ V(G) = E - N + 2P \]

Where:

- \(V(G)\) = Cyclomatic Complexity
- \(E\) = Number of edges in the control-flow graph
- \(N\) = Number of nodes in the control-flow graph
- \(P\) = Number of connected components, usually 1 for a single connected program

For a single connected flow graph, the formula becomes:

\[ V(G) = E - N + 2 \]

Another commonly used formula is:

\[ V(G) = D + 1 \]

Where \(D\) is the number of decision points.

### 6.1 Query-Handling Workflow

The chatbot's query-handling workflow can be represented using the following activities:

1. Start the chatbot.
2. Accept the user's query.
3. Identify the intent and extract entities.
4. Decide whether the query requires personalized information.
5. If personal information is required, authenticate the student.
6. If authentication succeeds, retrieve data from ERP/SIS.
7. If authentication fails, show an error and ask the student to retry.
8. If the query is general, search the knowledge base.
9. Display the response.
10. Decide whether the query is resolved.

11. If resolved, display the final answer.
12. If unresolved, offer clarification or create a support ticket.
13. End the workflow.

## 6.2 Decision Points

The representative workflow contains the following major decisions:

**Decision Condition**

Decision 1 Does the query require personal information?

Decision 2 Is authentication successful?

Decision 3 Is the query resolved?

Therefore, the number of decision points is:

$$ D = 3 $$

Using the decision-point formula:

$$ V(G) = D + 1 $$

$$ V(G) = 3 + 1 $$

$$ \boxed{V(G) = 4} $$

Thus, the simplified query-handling workflow has a cyclomatic complexity of 4, assuming these are the only decision points included in the modeled flow.

This indicates that at least four linearly independent paths should be considered when designing tests for this simplified workflow.

Note: This value is illustrative and based on the simplified workflow described above. The exact cyclomatic complexity of the implemented chatbot would depend on the actual source code and its complete control-flow graph.

## 6.3 Interpretation of Cyclomatic Complexity

| Complexity value | General interpretation |
|---|---|
| 1 | Straight-line logic without decision points |
| 2–5 | Relatively simple control flow |
| 6–10 | More complex control flow requiring additional testing |
| Above 10 | Potentially difficult to test and maintain |

These ranges are general guidelines, not strict rules. Complexity should be interpreted in the context of the function's purpose and implementation.

For the simplified chatbot workflow, a value of 4 indicates relatively simple control flow. However, individual functions such as authentication, NLP intent handling, and external API error handling may have higher complexity.

## 7. Function Point Analysis

Function Point Analysis (FPA) is a software size estimation technique that measures the functionality delivered to users, rather than counting lines of code.

It is useful when the programming language and implementation details are not yet known.

Function Point Analysis classifies functionality into five categories:

| Component | Meaning | Example in the chatbot |
|---|---|---|
| External Inputs (EI) | Data entering the system. | User queries, login details, ticket requests. |
| External Outputs (EO) | Processed information sent to users or external systems. | Chatbot answers, ticket confirmations, notifications. |
| External Inquiries (EQ) | Requests that retrieve information with limited processing. | Searching FAQs or checking ticket status. |
| Internal Logical Files (ILF) | Logical data groups maintained by the system. | Knowledge-base entries and ticket records. |
| External Interface Files (EIF) | Data groups maintained by external systems but referenced by the application. | Student records maintained in ERP/SIS. |

## 7.1 Function Point Formula

The unadjusted function point count is calculated as:

$$ UFP = \sum (\text{Count} \times \text{Weight}) $$

The weight depends on the component category and whether it is classified as simple, average, or complex.

For a traditional Function Point Analysis example, the following illustrative weights are used.

| Function type | Simple | Average | Complex |
|---|---|---|---|
| External Inputs (EI) | 3 | 4 | 6 |
| External Outputs (EO) | 4 | 5 | 7 |
| External Inquiries (EQ) | 3 | 4 | 6 |
| Internal Logical Files (ILF) | 7 | 10 | 15 |
| External Interface Files (EIF) | 5 | 7 | 10 |

## 7.2 Illustrative Function Point Calculation

For demonstration, assume the following preliminary counts for the chatbot. These are example estimates, not verified counts from the actual implementation.

| Function type | Assumed count | Assumed complexity | Weight | Total |
|---|---|---|---|---|
| External Inputs (EI) | 4 | Average | 4 | 16 |
| External Outputs (EO) | 3 | Average | 5 | 15 |
| External Inquiries (EQ) | 4 | Average | 4 | 16 |
| Internal Logical Files (ILF) | 2 | Average | 10 | 20 |
| External Interface Files (EIF) | 1 | Average | 7 | 7 |
| Total UFP | | | | 74 |

Therefore:

$$ UFP = 16 + 15 + 16 + 20 + 7 $$

$$ \boxed{UFP = 74} $$

The preliminary unadjusted function point estimate is 74 UFP, based on the assumed counts above.

The final function point value would require a detailed review of all user-visible functions, logical data groups, and their complexity.

## 7.3 Advantages of Function Point Analysis

- It is independent of the programming language.
- It can be used during early software planning.
- It focuses on functionality delivered to users.
- It can support approximate effort and cost estimation.

## 7.4 Limitations of Function Point Analysis

- It requires careful classification of functions and data groups.
- Different analysts may classify complexity differently.
- The estimate depends on the completeness of requirements.
- The assumed value cannot replace an actual count based on the final requirements and implementation.

# 8. Software Quality Metrics

Software quality metrics help determine whether the chatbot meets its expected requirements and provides a satisfactory user experience.

## 8.1 Response Time

Response time measures how long the system takes to respond after receiving a user query.

$$\text{Response Time} = \text{Response Completion Time} - \text{Request Time}$$

A lower response time generally provides a better user experience. Response time should be measured using actual requests under defined workloads.

### 8.2 Reliability

Reliability refers to the ability of the chatbot to perform its required functions without failure.

A simple availability formula is:

$$Availability(\%) = \frac{\text{Uptime}}{\text{Uptime}+\text{Downtime}} \times 100$$

For example, if a system operates for 990 minutes and experiences 10 minutes of downtime during a defined observation period:

$$Availability = \frac{990}{990+10}\times100$$

$$\boxed{Availability = 99\%}$$

This is an illustrative calculation and does not represent measured availability of the chatbot.

## 8.3 Maintainability

Maintainability measures how easily the software can be corrected, improved, or adapted.

For the chatbot, maintainability can be supported by:

- Separating the user interface from business logic.
- Keeping authentication separate from query processing.
- Using a dedicated knowledge-base service.
- Isolating ERP/SIS communication in an integration component.

- Maintaining clear documentation and automated tests.

### 8.4 Defect Density

Defect density measures the number of defects relative to the size of the software.

$$\text{Defect Density} = \frac{\text{Number of confirmed defects}}{\text{Software size in KLOC}}$$

Suppose a hypothetical implementation contains 5 confirmed defects and 2 KLOC of source code.

$$\text{Defect Density} = \frac{5}{2}$$

$$\boxed{2.5\text{ defects/KLOC}}$$

This example illustrates the calculation only. Actual defect density requires confirmed defects and the measured source-code size.

### 8.5 Test Coverage

Test coverage measures how much of the code or functionality is exercised by tests.

$$\text{Test Coverage}(\%) = \frac{\text{Items tested}}{\text{Total items}} \times 100$$

For example, if 45 out of 60 identified testable requirements have been tested:

$$\text{Coverage} = \frac{45}{60}\times100$$

$$\boxed{\text{Test Coverage} = 75\%}$$

This is an example calculation, not an actual test result.

Testing should cover general queries, successful and failed authentication, ERP/SIS errors, knowledge-base searches, ticket creation, ticket-status updates, and administrative access restrictions.

## 9. Metric Calculation Summary

The following table summarizes the example calculations and their limitations.

| Metric | Illustrative result | Interpretation |
|---|---|---|
| Lines of Code | 1,200 LOC assumed | Example only; source code must be counted. |
| Cyclomatic Complexity | 4 | Based on three decision points in the simplified workflow. |
| Function Points | 74 UFP | Based on assumed function counts and average complexity. |
| Availability | 99% | Example based on assumed uptime and downtime. |
| Defect Density | 2.5 defects/KLOC | Example based on 5 defects and 2 KLOC. |
| Test Coverage | 75% | Example based on 45 of 60 items tested. |

These values demonstrate how different software metrics are calculated. They must not be treated as actual project measurements unless supported by implementation and testing data.

## 10. Results and Interpretation

The analysis shows that different metrics provide different kinds of information about the AI Chatbot for College Helpdesk.

- Size metrics: LOC and Function Points can help estimate implementation size and development effort.
- Complexity metrics: Cyclomatic Complexity identifies independent execution paths and helps plan testing.
- Quality metrics: Response time, availability, defect density, and test coverage help evaluate the behavior and stability of the system.

- Maintainability: A modular architecture allows individual components to be tested and updated independently.

The simplified query-handling workflow has three major decisions, giving a cyclomatic complexity of 4 under the stated assumptions. The illustrative Function Point calculation produces 74 UFP. The other example metrics show how quality measurements can be calculated when actual project data becomes available.

A complete assessment should replace assumed values with measurements from the implemented chatbot, automated tests, logs, and monitoring tools.

## 11. Recommendations for Improvement

The following improvements can help maintain the quality of the chatbot:

1. Keep the chatbot's modules separate so that changes to one component do not unnecessarily affect others.
2. Add automated tests for general queries, authentication, personalized data retrieval, and ticket escalation.
3. Measure response time under normal and peak workloads.
4. Monitor external ERP/SIS failures and provide appropriate error messages.
5. Review complex functions and divide them into smaller, easier-to-test functions where appropriate.
6. Track defects during testing and calculate defect density after the software size is measured.
7. Maintain test cases for successful and unsuccessful user workflows.
8. Review logs and user feedback to identify unanswered queries and improve the knowledge base.

## 12. Conclusion

Software Metrics and Complexity Analysis provide a structured way to evaluate the size, complexity, quality, and maintainability of the AI Chatbot for College Helpdesk. Metrics such as LOC and Function Points help estimate software size, while Cyclomatic Complexity helps identify independent execution paths and plan testing.

The simplified query-handling workflow has an illustrative cyclomatic complexity of 4, and the example Function Point calculation produces 74 UFP based on assumed counts. Reliability, defect density, and test coverage can also be evaluated once actual implementation and testing data are available.

Overall, software metrics help developers identify complex areas, improve testing, maintain software quality, and support future enhancements to the chatbot.

## 13. References


1. Software Requirements Specification for AI Chatbot for College Helpdesk, Version 1.0.
2. Use Case Diagram for AI Chatbot for College Helpdesk.
3. UML Class Diagram for AI Chatbot for College Helpdesk.
4. UML Sequence and Activity Diagram Modeling Assignment.
5. McCabe, T. J., “A Complexity Measure,” IEEE Transactions on Software Engineering, 1976.
6. Standard software engineering concepts related to Function Point Analysis and software quality metrics.